Point Maps & Spatial Patterns

Workshop 5 — Module 7

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

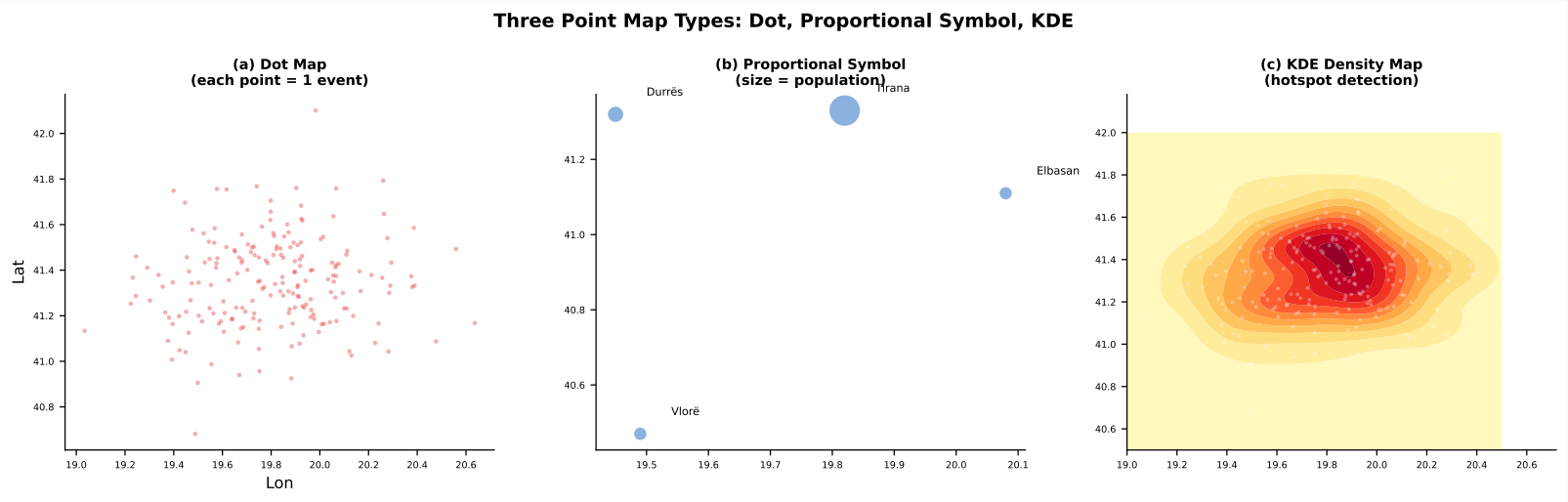
Module Roadmap

1. Three point map types: dot, proportional symbol, KDE
2. Overplotting: alpha, hexbin, 2D KDE contour
3. Proportional symbols: encoding a third variable
4. Spatial KDE for hotspot detection
5. Cartograms: distorting geography by data

Learning Outcome
You will produce dot maps for event locations, proportional symbol maps for magnitude, and KDE density surfaces for hotspot detection, handling overplotting with alpha, hexbin, or kernel smoothing.

Point Map Types

Dot, Proportional Symbol, KDE

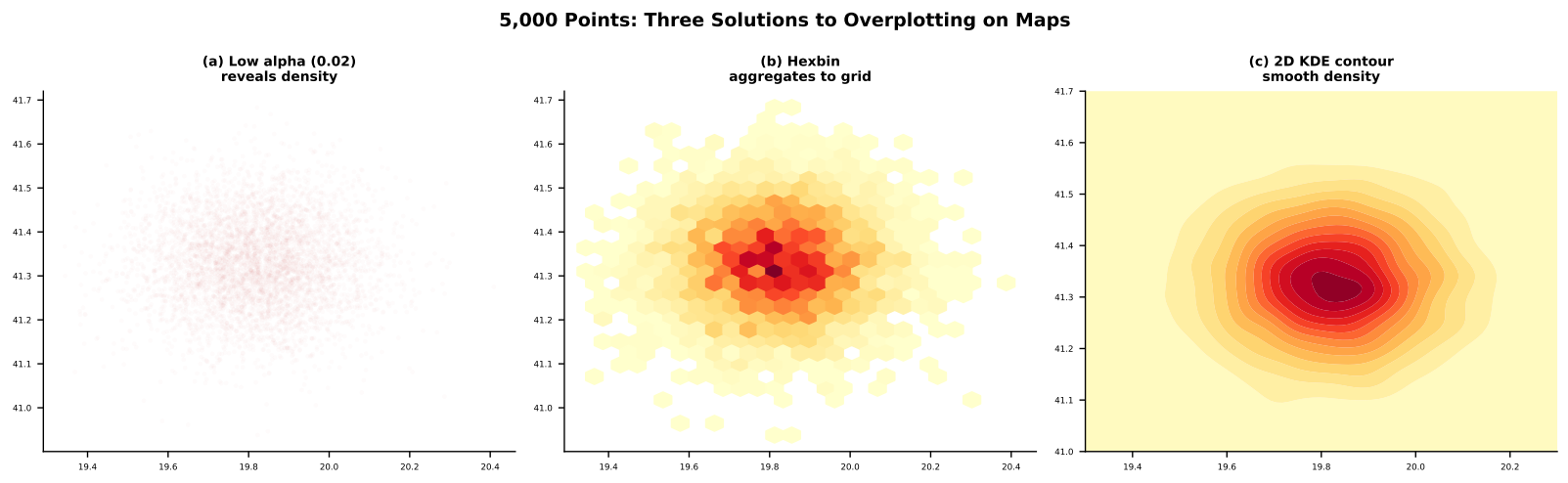


When to Use Which

Type	Encodes	Use When
Dot map	Location only	Event mapping (crimes, earthquakes)
Proportional symbol	Location + magnitude	City population, store revenue
KDE / heatmap	Density of events	Hotspot detection, spatial clustering
Hexbin	Binned count per cell	Large n (>5K points)

Overplotting

5,000 Points: Three Solutions



Pro-Tip

With >1,000 points, raw scatter becomes a blob. Three solutions ranked by effectiveness: (1) **alpha transparency** (cheap, approximate), (2) **hexbin** (exact counts per cell), (3) **2D KDE** (smooth continuous density). For maps, KDE is preferred because it produces a continuous surface interpretable as "event intensity per unit area."

Point Maps in Code

R

```r
# Dot map on sf geometry
ggplot(events_sf) +
  geom_sf(data = districts,
    fill = "grey95") +
  geom_sf(size = 0.5, alpha =
      0.3,
    color = "#E53935") +
  theme_void()
# Proportional symbol
ggplot(cities_sf) +
  geom_sf(data = districts) +
  geom_sf(aes(size = population),
    color = "#1565C0", alpha =
        0.5) +
  scale_size(range = c(1, 12))
# KDE overlay
ggplot(events_sf) +
  geom_sf(data = districts) +
  stat_density2d(aes(x = lon, y =
      lat,
    fill = after_stat(level)),
    geom = "polygon", alpha =
        0.5) +
  scale_fill_viridis_c()
```

Python

```python
# Dot map
fig, ax = plt.subplots()
districts.plot(ax=ax, color="
    grey95",
    edgecolor="grey70")
events.plot(ax=ax, markersize=1,
    color="#E53935", alpha=0.3)
# Proportional symbol
ax.scatter(cities.lon, cities.lat
    ,
    s=cities.pop/1000,
    c="#1565C0", alpha=0.5)
# KDE (scipy)
from scipy.stats import
    gaussian_kde
xy = np.vstack([events.lon,
    events.lat])
kde = gaussian_kde(xy)
xg, yg = np.mgrid[...]
z = kde(np.vstack([xg.ravel(),
    yg.ravel()])).reshape(xg.
        shape)
ax.contourf(xg, yg, z, levels=10,
    cmap="YlOrRd", alpha=0.7)
```

Cartograms

Cartograms: Geography Distorted by Data

Type	How	R/Python
Contiguous	Regions resized but connected	`cartogram` (R)
Dorling	Circles sized by data	`cartogram::cartogram_d`
Non-contiguous	Shrink/grow maintaining centroid	`cartogram::cartogram_n`
Tile grid	Each region = equal-size tile	Manual or `geogrid`

Data Insight

Cartograms solve the choropleth's biggest flaw: large-area, low-population regions dominate visually. By distorting area to match the data variable, cartograms give each region visual weight proportional to its importance.

Synthesis

Key Takeaways

1. **Dot maps** for event locations; **proportional symbols** for location + magnitude; **KDE** for density/hotspots.
2. **Overplotting** at $n > 1\text{K}$: use alpha, hexbin, or 2D KDE. KDE gives a continuous density surface.
3. **Cartograms** distort geography so area $\propto$ data, fixing the choropleth's large-region bias.
4. R: `geom_sf()` + `stat_density2d()` for static, `leaflet::addHeatmap()` for interactive.
5. Python: `geopandas.plot()` + `scipy.stats.gaussian_kde()` for static, `folium.plugins.HeatMap()` for interactive.

Essential Readings

- Lovelace, R. et al. (2019). *Geocomputation with R*, Chapter 8.
- Silverman, B. W. (1986). *Density Estimation for Statistics and Data Analysis*.
- Dorling, D. (1996). "Area Cartograms." *Concepts and Techniques in Modern Geography*.

Next Module: Interactive Maps (W05-M08)